

GraphTAN: Temporal Attention Network for Learning Graph-Level Embedding

Yan-Jiang Cai , Hai-Chun Cai , Chun-Yang Zhang , C. L. Philip Chen , *Life Fellow, IEEE*,
and Qian-Xi Tang 

Abstract—Numerous significant temporal graph tasks, such as graph similarity ranking, trend analysis and anomaly detection, necessitate low-dimensional and high-order graph-level embedding in terms of the evolving nodes and topologies over time. However, most existing graph embedding methods focus on extracting node-level embeddings, while ignoring these evolutions. Therefore, these methods inadequately consider the impact of trends on the overall graph. Moreover, there are a large number of nodes in temporal graphs, which make it difficult to generate effective graph embeddings directly by aggregating dynamic node embeddings. In this study, we propose a novel temporal attention network for learning graph-level embedding learning called GraphTAN. Specifically, the proposed model employs pooling attention to select crucial nodes and filter out noisy ones for each snapshot, thereby enhancing the quality of aggregated graph embeddings. Furthermore, we design a graph-level temporal attention mechanism to effectively extract temporal graph embeddings, capturing trends and patterns across snapshots. The proposed model is evaluated on three downstream tasks. Experimental results demonstrate that GraphTAN captures both the topology structure and fine-grained trend effectively, outperforming the state-of-the-art methods with big margins on multiple tasks over several real networks.

Index Terms—Anomaly detection, graph-level representation learning, graph neural networks (GNNs), temporal attention, trend analysis.

I. INTRODUCTION

GRAPH embedding techniques have extensive applications across various graph tasks, including graph classification and graph clustering, etc. Despite the significant advancements achieved in the field of static graph in recent years [1], [2], [3], there remains a considerable disparity between theoretical developments and real-world scenarios. In the real world, temporal graphs emerge from dynamic interactions among nodes in various contexts such as social networks [4], question-and-answer forums, or user-item interaction systems [5]. Many

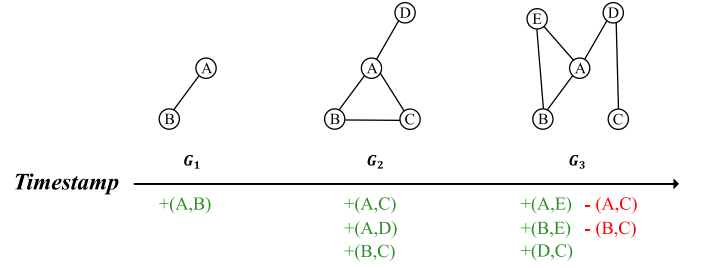


Fig. 1. Temporal graph example, where nodes and edges can be added or removed between time snapshots.

graph tasks require careful consideration of the temporal attributes associated with nodes and edges as they undergo additions, deletions, or changes over time as illustrated by the example in Fig. 1.

Recently, some work has explored temporal node and edge embedding methods [5], [6], [7], yet there remains a paucity of research focusing on the temporal graph-level embeddings [8], [9]. Such embeddings describe the global topology structure and node attributes of each snapshot, capturing the evolution patterns of the graph over time. Temporal graph embeddings are crucial for various valuable tasks, such as graph similarity [8], [9], [10] anomaly detection and trend analysis [11], [12]. The graph similarity task involves discerning similarities among snapshots. The objective is to find a snapshot that has the most similar structure to the anchor snapshot. This task has many application scenarios [13]. For example, when monitoring an organization with a history of illegal activity, it is possible to predict whether the organization is planning another crime by analyzing the communication graph, namely, comparing the similarity of the current snapshot to the target snapshot. The anomaly detection and trend analysis tasks involve identifying anomalous occurrences and trends within the temporal graphs. For instance, detecting protests in social networks, tracking unusual public opinion shifts generated by viral news stories, and monitoring structural changes in the company resulting from specific events.

Currently, numerous methodologies are available for obtaining static graph-level embedding. Some methods extract node embeddings individually and subsequently employ clustering methods [14], [15] to aggregate them into a single graph embedding [16]. Alternatively, contrastive learning methods [17] produce the optimal graph embeddings by comparing node

Received 1 July 2024; revised 24 October 2024 and 20 January 2025; accepted 19 February 2025. Date of publication 5 March 2025; date of current version 2 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62476059. (Corresponding authors: Hai-Chun Cai; Chun-Yang Zhang.)

Yan-Jiang Cai, Hai-Chun Cai, Chun-Yang Zhang, and Qian-Xi Tang are with the School of Computer and data Science, Fuzhou University, Fuzhou 350108, China (e-mail: fjsmche@163.com; zhangcy@fzu.edu.cn).

C. L. Philip Chen is with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, Guangdong 510006, China.

Digital Object Identifier 10.1109/TCSS.2025.3544460

and graph embeddings [18]. However, these studies predominantly focus on static graph representation learning, overlooking trends in graph-level embeddings [10], [19], which limit their ability to capture patterns necessary for temporal graph tasks. Subsequently, some existing node-level methods for temporal graphs focus on evolving nodes, while it is difficult to depict the dynamics of the graph by directly pooling node-level features. Existing works on learning temporal graph-level embeddings are limited, most methods simply rely on random walk techniques to transform temporal graphs into sequential data for learning graph embeddings. However, these methods inevitably lose essential temporal patterns due to the same sampling strategies, which cannot adapt to the rapid changes of temporal graphs. Additionally, they are incapable of processing the importance of nodes across different snapshots. In fact, the nodes may play different roles along with time, thus showcasing different contributions for the whole network. For example, a YouTuber uploading a trend-setting video may become a key node in the network temporarily, but their influence may significantly diminish at other times. This leads to a significant number of nodes whose importance varies over time, referred to as temporal noise nodes, which present challenges for learning effective temporal graph-level embeddings. Therefore, a tractable and learnable node selection mechanism is needed to effectively filter out temporal noise nodes. Moreover, it is also crucial to learn the temporal patterns of the graph.

Since the attention mechanism is proven to be effective in various fields [20], we propose a new temporal attention-based graph neural network (GNN) with a node selection mechanism to learn expressive temporal graph-level embeddings. In GraphTAN, temporal graph-level embeddings are produced through three attention mechanisms across different aspects: topology structure, node pooling, and graph-level dynamics. Structural attention extracts each node features from their local neighborhoods within each snapshot. Pooling attention employs the attention mechanism to determine which nodes should be removed. Temporal attention captures the evolution pattern of the graph across numerous timestamps by adaptively assigning weights to historical embeddings. In this way, the proposed model comprehends the complex topology structure and reveals the trends of each snapshot, and effectively representing trends, node attributes, and topology structure synchronously. The contributions of this work are summarized as follows.

- 1) A novel temporal attention GNN is proposed, which incorporates three attention mechanisms across three aspects. Unlike previous temporal graph embedding methods, GraphTAN synchronously captures trends, node attributes, and graph topology.
- 2) A modular architecture design for GraphTAN is proposed, enabling the flexible stacking of different layer types and facilitating computationally efficient learning compared with previous graph-based temporal methods.
- 3) Extensive experiments have been conducted across various tasks, such as graph similarity ranking, temporal anomaly detection, trend analysis, and visualization analysis. The results show that GraphTAN surpasses the state-of-the-art methods.

The rest of this article is organized as follows. Section II introduces several related works of graph representation learning. In Section III, we will introduce the proposed GraphTAN model in detail. The experimental results are reported in Section IV. Section V draws the conclusion and further works.

II. RELATED WORKS

Graph representation learning methods are primarily categorized into two types: node-level and graph-level methods. In static graph scenarios, they represent each node or graph with a single vector. Alternatively, temporal graphs involve the analysis of multiple snapshots, which entails generating a set of vectors for each node or snapshot, with a different number of nodes at each timestamp.

A. Static Node-Level Embedding

Matrix factorization-based methods involve decomposing graph-associated matrices, such as adjacency or Laplacian matrices, into lower-dimensional representations, while preserving key structural relationships of the graph. Techniques for static node embedding encompass matrix factorization-based methods, such as singular value decomposition (SVD) [21] and locally linear embedding (LLE) [22], which decompose the Laplacian matrix or higher-order adjacency matrices to generate node embeddings that preserve distances between nodes. Random walk-based methods specify different walking strategies to generate a sequence of nodes starting from every node on the graph, further maximizing the occurrence probabilities of all the nodes in the sequence when searching for their representations. Random walk-based methods such as DeepWalk [23] and node2vec [1] create random paths over the graph and produce node embeddings through a word2vec-like architecture [24]. Deep learning-based methods employ neural networks to learn graph representations by jointly encoding the topological structure and node attributes. In recent years, deep learning-based methods, particularly GNNs [16], have gained popularity, which includes graph convolutional networks (GCN) [2], graph attention networks (GAT) [25], and GraphSage [26]. These methods employ effective message-passing mechanisms to aggregate information between nodes, producing expressive node representations. However, traditional static graph methods are limited in their ability to handle temporal graphs, as they only capture static structural information and struggle to account for temporal variations.

B. Temporal Node-Level Embedding

A significant portion of existing research endeavors focuses on the depiction of node-level embeddings within the temporal domain. Part of these methods use static node embedding techniques directly, which are independently applied to temporal snapshots individually, followed by aligning them to produce a unified representation [27]. However, the alignment technique struggles to handle scenarios where nodes are added or removed over time. As a result, many methods use sequence models to deal with the temporal nodes.

Dynamic self-attention network (DySAT) [28] combines information from the structural neighborhood and temporal dynamics patterns through two attention modules to generate dynamic node representations. However, it has high model complexity, which is not applicable to large-scale graphs. Motif-preserving temporal shift network (MTSN) [29] introduces a dynamic GNN architecture that integrates network structure features and temporal patterns through a time-series migration mechanism. This design aims to capture the evolutionary patterns inherent in dynamic networks proficiently. DyG2Vec [30] introduces a window-based encoder–decoder model for temporal graphs. TREND [31] operates on temporal events and node dynamics, utilizing a GNN framework grounded in the Hawkes process. However, this process formulation does not scale for large graphs with millions of links.

While the aforementioned methods exhibit diversity, they all focus on temporal node-level representations. Consequently, aggregating node embeddings into graph-level embeddings is challenging, as these methods predominantly capture local node dynamics, making it difficult to effectively model the global temporal patterns of the entire graph.

C. Static Graph-Level Embedding

In graph-level methodologies, the emphasis is placed on analyzing and modeling the overall structure of the graph, rather than concentrating solely on individual nodes or local node relationships. UGraphEmb [10] involves embedding the entire graph into a vector space by utilizing the Graph Edit Distance (GED) metric [32] to preserve the proximity between graphs as the guidance for the training process. Hierarchical graph contrastive learning (HGCL) [33] proposes a hierarchical graph contrastive learning framework, which explores the hierarchical structural semantics of graphs at both node and graph levels. Meanwhile, Hi-PART [34] develops an effective pooling mechanism with a hierarchical partition tree, offering a significant improvement over the Weisfeiler–Lehman test. Graph2vec [19] conducts a static embedding of the entire graph without taking temporal factors into account, analogizing the graph to a document. In this analogy, rooted subgraphs surrounding each node are construed as contextual elements constituting the document. While static graph-level embedding methods effectively learn graph representations, they are challenging to apply directly to temporal graphs, as they fail to capture trends and overlook temporal patterns. Specifically, these methods treat each timestamp independently, without integrating the dynamics necessary for evolving graph embeddings.

D. Temporal Graph-Level Embedding

Temporal graph-level embedding methods differ significantly from node-level embedding methods. The latter focuses on capturing individual node changes, which can be managed with classical sequence models. In contrast, temporal graph-level tasks demand an extensive understanding of global changes across the entire graph. Temporal graph-level embedding involves assessing both the influence of nodes on the

overall graph structure and their relevance over time, presenting a more complex and challenging task. The tdGraphEmbed [8] leverages the Doc2Vec [35] architecture to produce temporal graph-level embeddings. Unlike traditional node embedding methods that use random walks, this approach globally embeds the entire graph structure and its nodes at each timestamp. Notably, it incorporates node attributes shared across different timestamps to capture the temporal patterns. In contrast, GraphERT [9] employs Transformer architecture [36] to embed the entire graph. It encodes each snapshot of the graph by conducting numerous random walks, which are fed into Transformer. By incorporating a temporal model to forecast the temporal indices of the random walks, the model captures both the contextual information of the paths and their temporal relationships.

Although existing methods have shown effectiveness, they rely on a two-stage process that separately learns snapshot features and temporal relationships, which struggle to adapt to rapidly changing snapshots and temporal node importance [37]. In addition, these methods apply a uniform strategy across all snapshots [30]. This strategy fails to capture the dynamic influence of nodes on the graph, potentially undermining structural integrity and incorporating unimportant nodes into graph-level embeddings [38]. Furthermore, these methods fail to account for variations in node importance over time, leading to a sub-optimal exploration of evolutionary patterns in temporal graphs from both structural and feature perspectives. This highlights the need for explicit modeling of evolution to more effectively capture both structural and temporal patterns.

III. PROPOSED MODEL

In this section, the main components of GraphTAN are introduced. As shown in Fig. 2, GraphTAN has three modules: structural attention module, pooling attention module, and temporal attention module. With these three different modules, efficient learning of temporal graph-level embeddings is achieved. The three modules collaborate to capture temporal patterns and can be stacked to construct GNNs with different complexity.

A. Problem Formulation

A temporal graph can be defined as a sequential arrangement of T graph snapshots: $G = \{G_1, G_2, \dots, G_T\}$. Each snapshot is denoted as $G_t = (V_t, E_t)$, where V_t represents nodes and E_t represents edges. Consequently, each snapshot encapsulates the graph's structure over a discrete timestamp.

Let V represent the set of all nodes present across all snapshots of G . Therefore, a subset of the vertex set V_t within G_t can be specified as $V_t \subseteq V$. Notably, the count of nodes at each time instance may vary from those in preceding instances, meaning that $|V_t| \neq |V_{t+1}|$.

Our goal is to learn the embedding h_G^t of the graph at each timestamp $t \in \{1, 2, \dots, T\}$, where $h_G^t \in \mathbb{R}^d$. The graph embeddings are expected to capture the temporal patterns of the graph, encompassing both its topological structure and node features, which can be effectively utilized in various downstream tasks.

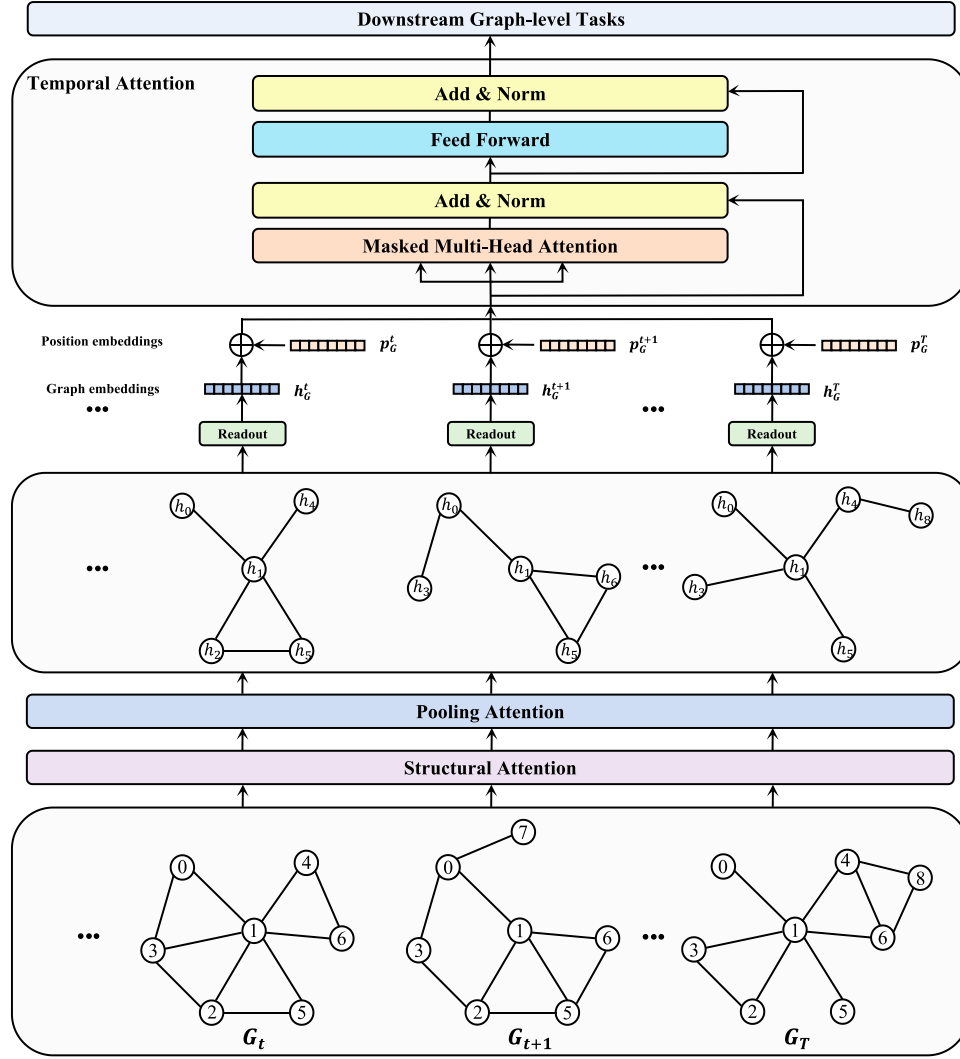


Fig. 2. Model framework of GraphTAN. The snapshots are processed by the structural attention layer, and upon obtaining the embeddings of the nodes corresponding to each snapshot, they are passed to the pooling attention layer. Subsequently, the graph-level embeddings are derived through the readout function, followed by combining the obtained embeddings with the positional information. Finally, these combined embeddings are input into the temporal attention layer to provide the final temporal graph-level embeddings for downstream tasks.

B. Structural Attention

Since the nodes between the various snapshots have different topologies, the model first needs to capture the higher-order node embeddings, which are then utilized to build the final graph embeddings. A structural attention layer is employed to process each snapshot $G_t \in G$, which consists of a set of node attributes $\{x_v \in \mathbb{R}^D, \forall v \in V_t\}$ and linking relations of the nodes. For each snapshot, the corresponding node embeddings $\{h_v \in \mathbb{R}^F, \forall v \in V_t\}$ can be obtained, to capture the node features and relationships between nodes in the snapshot G_t .

Given that the neighboring nodes have different importance for the target node, the structural attention layer assigns different weights for each neighborhood. The computation of neighborhood attention coefficients [25] e_{uv} between node u and node v is as

$$e_{uv} = \sigma(A_{uv}^t \cdot a^T [W_s x_u \parallel W_s x_v]), \forall (u, v) \in E_t \quad (1)$$

where $W_s \in \mathbb{R}^{D \times F}$ is the shared transformation weight applied to all nodes in each snapshot. The vector $a \in \mathbb{R}^{2D}$ represents a weight parameter used as an attention function; $\parallel$ denotes concatenation operation, and $\sigma(\cdot)$ is a nonlinear activation function. Note that A_{uv}^t signifies the weight of the link (u, v) within the snapshot G_t . Following calculating the coefficients between each node, the softmax function is employed to standardize the coefficients for convenient comparison, which is achieved by

$$\alpha_{uv} = \text{softmax}(e_{uv}) = \frac{\exp(e_{uv})}{\sum_{k \in \mathcal{N}_v} \exp(e_{ik})} \quad (2)$$

where α_{uv} indicates the importance of node u to node v , and $\mathcal{N}_v = \{u \in V_t : (u, v) \in E_t\}$ is the set of neighbors of node v in snapshot G_t .

Ultimately, the message propagation process is facilitated by the attention coefficient α_{uv} , which extracts the embedding h_v of each node. Given the extensive node count within G_t , all unlinked nodes possess α_{uv} values of zero. To address this,

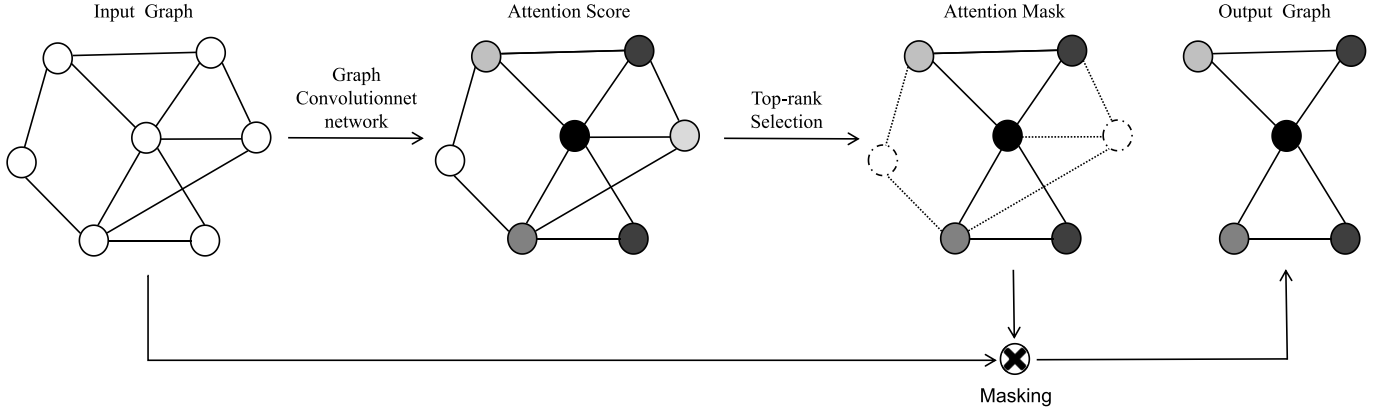


Fig. 3. Illustration of the pooling attention layer. Through graph convolution and activation functions, the attention score of each node can be obtained. The darker the node color, the higher the importance of the node. With these attention scores, the attention mask can be obtained for selecting nodes. The node features and graph structure can be masked to obtain new graph data.

sparse matrices are employed to implement masked attention towards neighboring nodes effectively as

$$h_v = \sigma \left(\sum_{u \in \mathcal{N}_v} \alpha_{uv} W_s x_u \right). \quad (3)$$

Through the structural attention layer, the embedding for each node can be obtained, and the node embedding matrix representation $H_t \in \mathbb{R}^{N \times F}$ of each snapshot G_t with N nodes can be derived.

C. Pooling Attention

For graph-level embeddings of static graphs, it is sufficient to learn a proper aggregation function. However, in temporal graphs, it is imperative to consider that as the pattern of the graph changes over time, an increasing number of noisy nodes may emerge that do not contribute to the graph embeddings. Therefore, it is necessary to consider the importance of different nodes to the whole graph at various timestamps. Pooling attention is employed to assign different weights to nodes for selecting the most suitable nodes in each snapshot. The pooling attention layer receives node embeddings $\{H_t \in \mathbb{R}^{N \times F}, \forall v \in V_t\}$ and edges E_t of each snapshot G_t , where F is the dimension of the input embeddings. Through pooling attention layer, the attention scores of each node can be computed. A subset of node embeddings and pooled node links can be obtained, where nodes that are more important to the graph-level representation are retained.

The pooling attention layer is employed to focus on all nodes within a snapshot. The specific pooling process is shown in Fig. 3. It uses an attention mechanism to compute the importance of each node, identifying whether a node is a temporal noise node or a relevant one, which serves the subsequent node selection process. Specifically, the proposed model uses graph convolution network [2] to derive the attention scores, which are computed as

$$Z = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^t W_{\text{pool}} \right) \quad (4)$$

where σ denotes the activation function, A is the adjacency matrix of the input snapshot, $\tilde{A} = A + I_N$, I_N is the matrix to represent the case of self-connection. Concurrently, $\tilde{D}$ represents the degree matrix of $\tilde{A}$, which is introduced to normalize the adjacency matrix. $W_{\text{pool}} \in \mathbb{R}^{F \times 1}$ is the model parameter of the pooling attention layer. This approach facilitates that the pooling outcomes are informed by a comprehensive understanding of the node features and topology structure. Various formulas for GNNs can be substituted into (4). The generalized formula for computing the attention score $Z \in \mathbb{R}^{N \times 1}$ is as

$$Z = \sigma \left(\text{GNN} \left(H^t, A^t \right) W_{\text{pool}} \right). \quad (5)$$

The node selection method in Graph U-Nets [39] is utilized, which adapts to graphs of varying sizes and identifies the more important nodes. The selected nodes are given by

$$\text{id} = \text{top} - \text{rank} \left(Z, \lceil kN \rceil \right), \quad Z_{\text{mask}} = Z_{\text{id}} \quad (6)$$

where $k \in [0, 1]$ is the pooling ratio, serving as the hyperparameter that determines the number of nodes preserved. The process of selection identifies the top $\lceil kN \rceil$ nodes based on the attention score matrix Z . $\text{top} - \text{rank}$ is a function that returns the index of the top $\lceil kN \rceil$ scores, $\cdot_{\text{id}}$ represents an indexing operation, and Z_{mask} denotes the feature attention mask matrix. New graph data can be obtained with Z_{mask} , and it is defined as

$$H^{t'} = H_{\text{id}}^t, \quad H_p^t = H^{t'} \odot Z_{\text{mask}}, \quad A_p^t = A_{\text{id}, \text{id}}^t \quad (7)$$

where H_{id}^t is the row-wise indexed identity matrix, $\odot$ denotes the broadcasted elementwise product, and $A_{\text{id}, \text{id}}^t$ represents the row-wise and column-wise indexed adjacency matrix. The matrices H_p^t and A_p^t correspond to the updated feature matrix and the associated adjacency matrix, respectively.

The original graph can be dynamically selected via the pooling attention layer. Through end-to-end training, the model is able to select nodes that are important for graph representation or nodes that reflect temporal patterns.

D. Temporal Self-Attention

By pooling attention layer, the more refined subgraphs of each snapshot can be generated, which contain only the key

nodes and drops the noise nodes. To enable the pooling attention layer to pay attention to temporal important nodes, it is necessary to aggregate the nodes to generate the temporal embeddings of each snapshot subgraph. The embeddings of each snapshot subgraph are obtained through a readout function, denoted as $H_G^t = \{H_G^1, H_G^2, \dots, H_G^T\} \in \mathbb{R}^{D'}$, where T represents the total number of timestamps.

Moreover, to further capture evolutionary patterns in temporal graphs, a temporal self-attention layer is developed. With the learnable subgraph embeddings, the proposed model utilizes a temporal self-attention mechanism to further capture temporal patterns between snapshots. Temporal self-attention aims to learn a sequence of embeddings of the graph G at each timestamp, i.e., $z_G^t = \{z_G^1, z_G^2, \dots, z_G^T\}$, $z_G^t \in \mathbb{R}^{F'}$.

The H_G^t is used as a query to focus on its historical embeddings ($< t$), tracking the evolution of the graph G_t over time. Temporal attention relies on the history of the current snapshot, enabling the learning of graph structural information within timestamps and more in-depth exploration of the graph's historical context.

The scaled dot-product form [36] of attention is utilized to compute the representation of a graph H_G^t , where keys and values are set to the graph-level embeddings at all other timestamps. Queries are set to the graph-level representation at moment t . This self-attention mechanism effectively captures dependencies between different graph snapshots, allowing the model to focus dynamically on relevant temporal information, which is essential for representing evolving graph structures. It is important to note that the snapshot embedding at timestamps t is determined in relation to previous graph embeddings, allowing each timestamp t to attend to timestamps ranging from 0 to t . A temporal self-attention coefficient is defined as

$$e^{ij} = \left(\frac{((H_G^i W_q) (H_G^j W_k)^T)}{\sqrt{F'}} + M_{ij} \right) \quad (8)$$

where the ij refer to the snapshots G_i and G_j respectively, $W_q \in \mathbb{R}^{D' \times F'}$, $W_k \in \mathbb{R}^{D' \times F'}$ are linear projection matrices, $M \in \mathbb{R}^{T \times T}$ is a mask matrix, which is defined as

$$M_{ij} = \begin{cases} 0, & i \leq j \\ -\infty, & \text{otherwise} \end{cases} \quad (9)$$

when $M_{ij} = -\infty$, softmax leads to zero attention weights. It functions similarly to masking attention, preventing the current snapshot from accessing information from future snapshots. The notation β_t^{ij} denotes the attention transfer from timestamp i to j . The softmax function is used to normalize the attention coefficients of the graph embedding across all timestamps, which is given by

$$\beta^{ij} = \frac{\exp(e^{ij})}{\sum_{k=1}^T \exp(e^{ik})}. \quad (10)$$

Finally, the temporal graph embeddings z_G^t for each snapshot are determined by calculating a linear combination of the

features associated with their respective snapshots, and it is defined as

$$z_G^t = \sum_{j=1}^t \beta^{tj} (H_G^j W_v) \quad (11)$$

where $W_v \in \mathbb{R}^{D' \times F'}$ is a linear projection matrix that transforms values into different spaces.

E. GraphTAN Architecture

In this section, the overall framework of GraphTAN is introduced, which is a new temporal GNN architecture developed for learning graph-level embeddings. GraphTAN incorporates structural attention, pooling attention, and temporal self-attention layers as fundamental modules. The model is designed to process a collection of snapshots with the objective of learning graph-level embeddings for each snapshot.

The structural attention layer computes node embeddings for each snapshot by extracting features from the local neighborhood of each node. These node embeddings are fed into the pooling layer, which computes the importance of each node through attention scores and selects the nodes that are more important for each snapshot. Following the selection of nodes, the readout function is used for generating graph-level embeddings, serving as the input for the temporal attention layer. This layer is designed to concentrate on historical snapshots to capture temporal patterns in the graph structure. The temporal attention layer has the capability to provide temporal graph-level embeddings and is designed to enhance adaptability to dynamic changes within the graph-level embeddings. The integration of structural and temporal patterns allows models to better capture the temporal evolution of interactions and their impact on graph attributes, resulting in more accurate predictions and deeper insights into temporal graphs.

The maximum common subgraph (MCS) metric [40] is capable of capturing the similarity between different snapshots and is widely adopted in graph search. Through GraphTAN, the graph embedding of the G_i and G_j can be obtained, denoted as z_G^i and z_G^j , respectively. The loss function can be defined as

$$\begin{aligned} \ell &= E_{(i,j) \sim D} (s_{ij} - s_{ij}) \\ &= E_{(i,j) \sim D} \left((z_G^i)^T z_G^j - s_{ij} \right)^2 \end{aligned} \quad (12)$$

where s_{ij} represents the calculated MCS metric between G_i and G_j . Additionally, notice that the learning is performed in a self-supervised manner, without having graph labels. By minimizing the loss function, GraphTAN is enabled to model the entire graph structure and its evolutionary patterns, thus learning expressive graph-level embeddings. The loss function can also be adapted for different downstream tasks by making appropriate adjustments.

Temporal graphs in real-world scenarios typically undergo changes that are shaped by a variety of contributing factors. For example, changes in user preferences for films of different genres (e.g., science fiction or comedy) may exhibit different temporal patterns. GraphTAN is an end-to-end model where nodes selected by the pooling attention layer are passed into

TABLE I
STATISTICS OF DATASETS

Dataset	#Nodes	#Edges	#Steps	#Resolution
Facebook	46 873	857 815	30	Monthly
Enron	87 062	1 146 800	182	Weekly
Game of Thrones	156 732	834 753	62	Daily
Formula1	38 702	254 731	61	Daily

the temporal self-attention module, which extracts temporal features. The model is trained with a loss function that integrates temporal information, and during backpropagation, the temporal correlations propagate from the temporal self-attention module to the pooling attention layer. This enables the pooling mechanism to access temporal dependencies across snapshots, helping it identify and filter out noisy nodes that are less relevant in the temporal aspect. As a result, our model is equipped with the capability to capture the evolution of temporal graphs from various perspectives by properly employing attention mechanisms.

IV. EXPERIMENT

In this section, the effectiveness of the proposed GraphTAN is evaluated on four large real-world social networks. By comparing the results of similarity ranking and anomaly detection tasks with state-of-the-art models, it is confirmed that the embeddings generated by GraphTAN demonstrate superior performance across multiple tasks.

In the experiments, all graph snapshots as well as nodes at the current timestamp are available, excluding newly emerging nodes that have not been previously observed. This setup ensures a controlled environment for evaluating the effectiveness of our approach.

A. Datasets

Four large real-world social networks are used in the following experiments, including Facebook wall posts, the Enron email dataset, the Reddit Game of Thrones community, and the Reddit Formula1 community. A comprehensive statistical overview of these networks is delineated in Table I, which describes the number of nodes and edges, the time step and the temporal resolution of each temporal graph.

- 1) *Facebook wall posts* is a social network, nodes represent users, and directed edges represent that user A posts on the wall of user B at a given timestamp, spanning August 2006 to January 2009 with monthly resolution.
- 2) *Enron* is the company's email network that captures all internal communications, with temporal edges denoting emails from user A to B at time t . The timestamps are weekly from January 1999 to June 2002.

- 3) *Reddit "Game of Thrones"* includes daily interaction graphs for 62 days, from July 1 to August 31, 2017, based on user comments and replies.
- 4) *Reddit "Formula1"* with graphs constructed at a daily level of detail for 61 successive days, from March 1 to April 30, 2019.

B. Baselines

We conducted comprehensive experiments with a variety of state-of-the-art benchmarks, encompassing both node-level and graph-level embedding methods.

For the static node embedding approach, node changes over time are captured by aligning each node's embedding to the same feature space on each snapshot. The graph embeddings are derived through node embeddings, by utilizing the conventional average pooling function. The following node-level embedding techniques are taken into account.

- 1) *Node2vec (2016)* [1] employs the node2vec algorithm on each snapshot and subsequently adjusts the embeddings through rotation to achieve alignment [27].
- 2) *SDNE (2016)* [3] employs the SDNE algorithm on each snapshot and subsequently adjusts the embeddings through rotation to achieve alignment [27].
- 3) *GAE (2016)* [41] implements GNN with mask autoencoder framework on graph data, using GCNs as encoder and decoder. GAE is applied to each snapshot and subsequently adjusts the embeddings through rotation to achieve alignment [27].
- 4) *DynamicTriad (2018)* [42] utilizes the Dynamic Triad to extract dynamic graph node embedding, ensuring the preservation of both structural and evolutionary graphs.
- 5) *Dynamic SDNE (2018)* [11] employs deep autoencoders in a manner akin to SDNE, progressively creating node embeddings for a graph at time t , initiating with the node embeddings at time $t - 1$.
- 6) *DynAERNN (2020)* [6] initiates the process with a fully connected encoder to extract a low-dimensional, hidden representation, which is subsequently fed through an LSTM network and a fully connected decoder.
- 7) *EvolveGCN (2020)* [43] uses GCN to generate node embeddings on each snapshot. These embeddings are subsequently input to a recurrent architecture to capture temporal dynamics. The model is trained for a link prediction task, and node embeddings are extracted at each timestamp.
- 8) *MTSN (2021)* [29] incorporates network motif into a temporal transition framework, which adeptly captures the developmental trends within evolving networks. The model is trained on link prediction task, and node embeddings are extracted at each timestamp.
- 9) *TREND (2022)* [31] is driven by temporal events and node dynamics, utilizing a Hawkes process-based GNN. The model is also trained on link prediction task, and the node embeddings are derived from each timestamp.

TABLE II
HYPERPARAMETER SETTINGS IN THE EXPERIMENTS

Dataset	Facebook	Enron	Game of Thrones	Formula 1
lr	0.01	0.015	0.015	0.01
λ	0.0005	0.0005	0.0005	0.0005
d	256	256	256	256
k	0.7	0.7	0.5	0.7
σ	ELU	ELU	ELU	ELU

For graph-level embedding methods, several baseline models are used, which include both static graph and temporal graph approaches.

- 1) *Graph2vec* (2017) [19] learns data-driven distributed representations of arbitrary-sized graphs. Initially designed for static graphs, the implementation has been extended to the temporal snapshots of the graph.
- 2) *Sub2vec* (2018) [44] generates embeddings for each sub-graph by utilizing random walk to encapsulate the sub-graph's node feature and topology structure.
- 3) *UGraphEmb* (2019) [10] learns graph-to-graph proximity using GED metric as ground truth. A refined model has been deduced based on the WEB dataset.
- 4) *tdGraphEmbed* (2020) [8] is a random-walk-based method inspired by Doc2Vec, which creates temporal embeddings at the graph level and improve random-walk based node embedding techniques to incorporate global embeddings for both nodes and the entire graph at each timestamp.
- 5) *GraphERT* (2023) [9] utilizes transformer architecture to embed the entire graph. It encodes each snapshot through numerous random sequences, feeding them into Transformer, which then provides global embeddings for each snapshot. Its reliance on random sequences can lead to inefficiencies in capturing important temporal dynamics.

The hyperparameter settings of the proposed model are shown in Table II. lr is the learning rate, λ denotes the weight decay controls the model complexity. d is the number of the hidden units; k represents the pooling ratio, and σ denotes the activation function. As shown in Table II, the hyperparameters used in the experiments are mostly same across the four datasets, except for the pooling ratio k , which decreases as the dataset size increases.

C. Similarity Ranking

In previous graph embedding methods [8], [9], the similarity ranking task is a common verification manner to evaluate whether the graph-level embeddings preserve graph-to-graph similarities. The task involves determining the similarity between a specified target snapshot and other snapshots, to identify the graph structure that exhibits the highest similarity. This necessitates that the graph embeddings effectively capture the features of snapshots and the interrelations among them. The cosine similarity metric is employed to evaluate the similarity ranking by calculating the similarity of graph embeddings at each timestamp.

The ranking performance is evaluated using rank-aware evaluation metrics, with Precision@ K defined as

$$\text{Precision@}K = \frac{\#Hits}{K} \quad (13)$$

where K denotes the number of top suggested items. $\#Hits$ is the number of snapshots that are most similar to the current snapshot. The MRR (mean reciprocal rank) is given by

$$\text{MRR} = \frac{1}{|G|} \sum_{G_t \in G} \frac{1}{r_{G_t}} \quad (14)$$

where $|G|$ is the total number of graph snapshots, and r_{G_t} is the rank of the first ground truth similar graph to G_t . The MAP@ K (mean average precision) [45] is defined as

$$\text{MAP@}K = \frac{\sum_{G_t \in G} \frac{\sum_{k=1}^K P(k) \text{rel}(k)}{\#Hits@K}}{|G|} \quad (15)$$

where $P(k)$ is the precision at cut-off k in the list, $\text{rel}(k)$ equals 1 if the snapshot at rank k is a relevant similar graph, and 0 otherwise.

The similarity ranking results presented in Table III indicate that our method outperforms other baseline methods in capturing graph proximity on the Facebook, Reddit-Game of Thrones, Reddit-Formula, and Enron datasets.

As shown in Table III, the performance of the temporal node-level method is inferior to that of graph-level methods. Moreover, the node-level methods have abnormal performance fluctuation in different datasets, which shows that the node-level methods are difficult to capture the global graph-level embeddings effectively. In addition, when the time resolution is large, the differences between the snapshots become more pronounced, allowing the model to more easily capture significant temporal features. Conversely, in Facebook dataset, the improvement in Precision@10 is less significant. As the time granularity decreases and the intervals between snapshots become shorter, it becomes more challenging to capture discriminative temporal features. This is evident in the Game and Formula1 datasets, where other models tend to underperform, but GraphTAN effectively captures these temporal distinctions, achieving superior performance. Furthermore, it can be seen that there is also a significant improvement in the results on the Enron dataset, which is due to the fact that the Enron dataset has more snapshots and smaller temporal resolution, and the proposed model captures this variation well and therefore has better performance. By considering both structural and temporal features simultaneously, GraphTAN achieves better performance, particularly on more complex tasks.

To better illustrate the significance of the graph-to-graph cosine similarity score, Fig. 4 shows the heatmaps of graph-to-graph cosine similarity scores in two datasets. For the Facebook dataset in Fig. 4(a), a cyclic pattern is observed on the diagonals at 12-month and 24-month intervals, suggesting a recurring annual structure. This pattern is governed by the annual periodic events such as anniversaries and birthday greetings that often appear on the user's wall, which shows that GraphTAN can effectively identify similar patterns across different snapshots. As shown in Fig. 4(b), the similarity matrix between the company's

TABLE III
EXPERIMENTAL RESULTS OF SIMILARITY RANKING

Model	Facebook				Enron			
	Precision@5	Precision@10	MRR	MAP@10	Precision@5	Precision@10	MRR	MAP@10
Node2vec	0.600	0.670	0.416	0.779	0.285	0.366	0.272	0.478
SDNE	0.220	0.393	0.202	0.402	0.043	0.093	0.048	0.091
GAE	0.480	0.603	0.373	0.727	0.237	0.288	0.166	0.395
DynamicTriad	0.677	0.743	0.483	0.840	0.234	0.323	0.250	0.408
Dynamic SDNE	0.386	0.550	0.207	0.534	0	0.005	0.022	0
DynAERNN	0.220	0.370	0.177	0.372	0.024	0.051	0.030	0.054
EvolveGCN	0.560	0.703	0.592	0.349	0.376	0.431	0.313	0.506
MTSN	0.158	0.297	0.123	0.327	0.054	0.116	0.042	0.157
TREND	0.640	0.723	0.383	0.833	0.197	0.274	0.129	0.274
Graph2vec	0.220	0.433	0.162	0.506	0.032	0.041	0.038	0.069
Sub2vec	0.293	0.557	0.194	0.476	0.042	0.055	0.089	0.102
UGraphEmb	0.567	0.750	0.268	0.815	0.107	0.162	0.122	0.204
tdGraphEmbed	0.660	0.760	0.392	0.831	0.272	0.369	0.306	0.445
GraphERT	0.726	0.810	0.615	0.849	0.452	0.463	0.467	0.595
GraphTAN	0.786	0.827	0.690	0.855	0.508	0.568	0.518	0.665

Model	Reddit-Game of Thrones				Reddit-Formula 1			
	Precision@5	Precision@10	MRR	MAP@10	Precision@5	Precision@10	MRR	MAP@10
Node2vec	0.310	0.338	0.358	0.542	0.167	0.206	0.204	0.376
SDNE	0.252	0.346	0.305	0.393	0.151	0.231	0.201	0.273
GAE	0.224	0.267	0.249	0.409	0.141	0.204	0.195	0.333
DynamicTriad	0.282	0.301	0.402	0.487	0.190	0.231	0.208	0.395
Dynamic SDNE	0.300	0.316	0.330	0.494	0.141	0.201	0.101	0.289
DynAERNN	0.087	0.166	0.088	0.196	0.095	0.203	0.081	0.255
EvolveGCN	0.138	0.211	0.107	0.267	0.203	0.257	0.166	0.366
MTSN	0.097	0.162	0.074	0.221	0.093	0.203	0.131	0.224
TREND	0.180	0.295	0.192	0.355	0.121	0.226	0.109	0.282
Graph2vec	0.190	0.274	0.185	0.329	0.082	0.168	0.059	0.215
Sub2vec	0.087	0.193	0.124	0.168	0.032	0.114	0.042	0.149
UGraphEmb	0.245	0.288	0.225	0.413	0.170	0.256	0.122	0.308
tdGraphEmbed	0.351	0.358	0.450	0.561	0.229	0.273	0.257	0.429
GraphERT	0.406	0.415	0.485	0.611	0.246	0.277	0.265	0.432
GraphTAN	0.483	0.479	0.527	0.623	0.301	0.329	0.299	0.436

Note: The bold entries indicates the best performance.

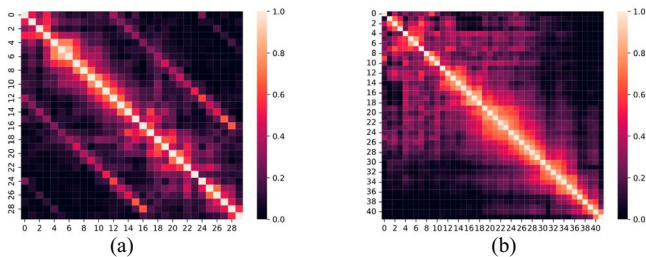


Fig. 4. Graph-to-graph similarity heatmap. (a) Facebook dataset over 30 months, with a cyclic diagonal in 12 months and 24 months indicating the graph structure is repeating. (b) Enron company's email network over 42 months, where we have strong similarity at Enron company over 42 months, revealing notable parallels between the initial and final periods of the company.

email networks is roughly divided into four areas, where the upper left corner and low right corner are highlighted. It implies that the temporal company's email networks are divided into

two stages, corresponding to the establishment and dissolution. The main diagonals of both figures are highlighted, which reflects that GraphTAN can reflect the similarity of neighboring snapshots, namely, it can effectively preserve the historical structures and features. Overall, the embeddings can be used to efficiently calculate the similarities between snapshots and restore the temporal patterns in the original data.

D. Anomaly Detection

The objective of anomaly Detection is to identify unusual occurrences within the temporal graphs. Anomalies in temporal graphs are defined as structural alterations that surpass the average scale. These changes can result from the addition or removal of nodes and edges, the movement of multiple nodes between communities, the emergence of new hubs, etc. Given

TABLE IV
EXPERIMENTAL RESULTS OF ANOMALY DETECTION

Model	Reddit- Game of Thrones		Reddit- Formula 1	
	MRR	Spearman	MRR	Spearman
Node2vec	0.370	0.559	0.129	0.449
SDNE	0.034	−0.336	0.037	−0.070
GAE	0.056	0.630	0.327	0.585
DynamicTriad	0.239	0.634	0.425	0.424
Dynamic SDNE	0.281	0.134	0.085	0.502
DynAERNN	0.072	−0.098	0.027	−0.396
EvolveGCN	0.061	0.279	0.191	−0.112
MTSN	0.062	0.031	0.061	0.270
TREND	0.041	−0.458	0.051	0.233
Graph2vec	0.332	0.647	0.136	0.424
Sub2vec	0.106	0.534	0.115	0.413
UGraphEmb	0.364	0.365	0.038	0.252
tdGraphEmbed	0.355	0.751	0.508	0.485
GraphERT	0.370	0.782	0.521	0.556
GraphTAN	0.576	0.849	0.560	0.770

Note: The bold entries indicates the best performance.

the absence of labels for the graph snapshots, this task serves as a substitute for graph classification, aiming to categorize snapshots as anomalies. The $\Delta G@t$ is defined as the change in the graph embeddings between two consecutive timestamps $t - 1$ and t [8]. Intuitively, values of $\Delta G@t$ that are higher than a predefined threshold can serve as indicators of anomalies. The cosine similarity is used to define $\Delta G@t$ as

$$\Delta G@t = \frac{\phi(G_t) \cdot \phi(G_{t-1})}{\|\phi(G_t)\| \cdot \|\phi(G_{t-1})\|} \quad (16)$$

where $\phi(G_t)$ represents the embedding of the graph at timestamp t .

In the “game of thrones” and “formula 1” datasets derived from Reddit, anomalies corresponding to the airing dates of the show’s episodes and the dates of formula 1 races have been labeled. These specific dates are known to trigger significant peaks in user activity, and it is crucial for the embedding technique to effectively capture such events. The “game of thrones” dataset has seven anomalies, and the “formula1” dataset has four. The MRR and spearman metric are employed to measure the model performance.

Anomaly detection requires the model to learn efficient embeddings that represent dynamic changes in the graph. As shown in Table IV, graph-level methods are significantly superior to node-level methods. Besides, GraphTAN surpasses competing methods, reflecting that the node selection mechanism can effectively filter noisy nodes to extract expressive

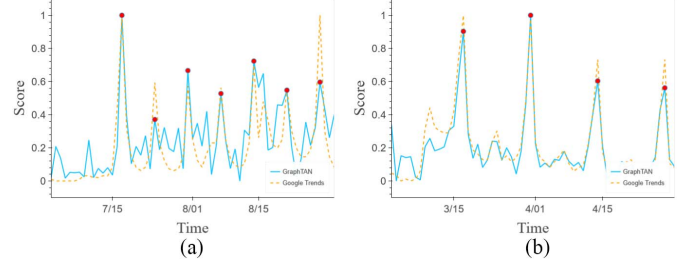


Fig. 5. $\Delta G@t$ over time of GraphTAN (blue, solid line) versus Google trends (orange, dashed line); the red points are the ground-truth anomalies. (a) Reddit-Game of thrones. (b) Reddit-formula 1.

embeddings of each snapshot and make it easier to identify abnormal snapshots.

E. Trend Analysis

In this study, graph embeddings are employed for detecting patterns in dynamic networks. The analysis relies on Google Trends, which is a popular tool for monitoring the prevalence of search terms. Given its broad monitoring of numerous queries, it is considered a trustworthy gauge of public curiosity. A surge in public interest is regarded as a “ground truth” for identifying trends. We evaluated the correlation between Google trends and our embedding trend, $\Delta G@t$ defined in (16), using the Spearman correlation metric [46]. The trend analysis is conducted on the two Reddit datasets, as they pertain to subjects that are searchable and trackable on Google trends.

The results of the trend analysis task are detailed in Fig. 5. The metrics are normalized using the Min-Max scaler to standardize them on a scale from 0 to 1. Fig. 5(a) and 5(b) highlights the pronounced correlation observed between the GraphTAN model and the anomalies pinpointed by Google Trends. The results show that GraphTAN has a highly significant correlation with Google Trends. This demonstrates that GraphTAN can effectively capture the temporal patterns and depict the differences between various snapshots through three attention mechanisms. Especially in Reddit-Formula 1, when there are relatively regular changes, the proposed temporal attention mechanism can be quite accurate in identifying the evolution of the graph pattern.

F. Ablation Experiment

To verify the effectiveness of the pooling attention and temporal self-attention modules, a series of ablation experiments are conducted. Specifically, the model performance is tested without the structural attention module, pooling attention module, and temporal self-attention module, denoted as GraphTAN-NS, GraphTAN-NP, and GraphTAN-NT, respectively. We also use random walk to replace the structural attention module and pooling attention module, called Graph-RW. Specifically, it uses random walk to generate control embeddings, which are fed into the temporal self-attention module to produce the final graph-level embeddings. For the similarity ranking task, the Facebook dataset is selected for the ablation experiments, and the results are presented in Table V. Furthermore, the Reddit-Game of

TABLE V
ABLATION EXPERIMENT ON SIMILARITY RANKING USING
FACEBOOK DATASET

Model	Precision@5	Precision@10	MRR	MAP@10
GraphTAN	0.786	0.827	0.690	0.855
GraphTAN-NP	0.627	0.783	0.367	0.841
GraphTAN-NT	0.280	0.410	0.161	0.404
GraphTAN-NS	0.260	0.417	0.245	0.452
GraphTAN-RW	0.706	0.753	0.489	0.839

Note: The bold entries indicates the best performance.

TABLE VI
ABLATION EXPERIMENT ON ANOMALY DETECTION
USING GAME OF THRONES DATASET

Model	MRR	Spearman
GraphTAN	0.576	0.849
GraphTAN-NP	0.447	0.726
GraphTAN-NT	0.219	0.458
GraphTAN-NS	0.140	0.242
GraphTAN-RW	0.250	0.631

Note: The bold entries indicates the best performance.

Thrones dataset is chosen for the anomaly detection task, with the results shown in Table VI.

As shown in Tables V and VI, GraphTAN-NP exhibits a noticeable performance drop. These results may be because the temporal graph contains a significant amount of temporal noise nodes, which can negatively impact the model performance when removing the pooling attention module. Additionally, the substantial decline of GraphTAN-NT and GraphTAN-NS highlights the importance of learning both local structural features and global temporal patterns when modeling temporal graphs. Furthermore, GraphTAN-RW demonstrates significant differences between the two datasets, which can be attributed to more complex temporal variations in the Game of Thrones that has more snapshots, whereas GraphTAN, an end-to-end model, consistently maintains stable performance across different datasets.

G. Statistical Analysis

Fig. 6 presents the results of the Friedman test for statistical analysis among different models, utilizing the results from six experiments presented in Tables IV and V. Specifically, the horizontal coordinate at the midpoint of each line represents the average ranking of each model. The length of line represents the critical difference, which is calculated by

$$CD = q_{\alpha} \sqrt{\frac{k(k-1)}{6N}} \quad (17)$$

where k represents the number of models, N is the number of experiments, and q_{α} denotes critical values. When α is set as 0.05, the critical value q_{α} is 2.850, and the critical difference CD is 2.124. Fig. 6 shows that the proposed method significantly outperforms the temporal node-level methods, namely

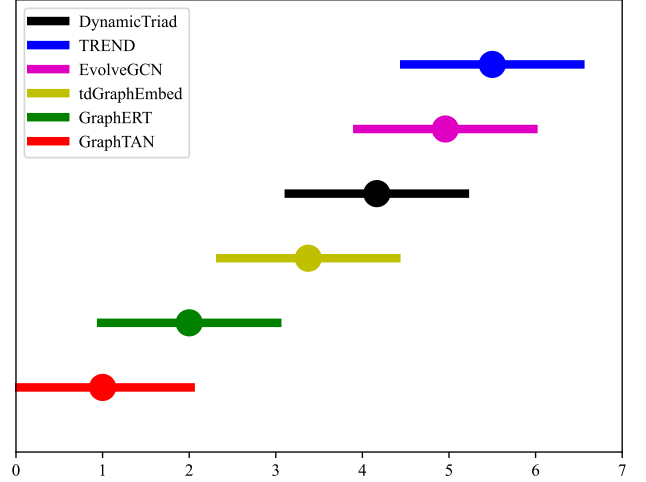


Fig. 6. Analysis results of the Friedman test.

TABLE VII
COMPLEXITY ANALYSIS

Models	Complexity	Time (min)
UGraphEmb	$O(ETd)$	0.4
tdGraphEmbed	$O(\gamma LNT + \gamma(N + w)NTd)$	6.6
GraphERT	$O(\gamma LNT + \gamma NL^2Td)$	2.6
GraphTAN	$O(ETd + pNTd + T^2d)$	0.9

DynamicTriad, TREND, and EvolveGCN, while surpassing the temporal graph-level methods tdGraphEmbed and GraphERT.

H. Complexity Analysis

The main computational cost of GraphTAN is composed of three attention modules. We assume that the output dimension of each layer is d . For structural attention layer, the complexity of calculating H_t is $O(ETd)$, where E represents the number of edges, and T denotes the number of snapshots. The complexity of the pooling attention layer and temporal self-attention layer is $O(NTd + T^2d)$, where N represents the number of nodes in each snapshot. Therefore, the overall complexity of GraphTAN is $O(ETd + pNTd + T^2d)$.

For tdGraphEmbed and GraphERT, γ is the number of samples per node, and $\mathcal{L}$ indicates the length of random walk sequences. w is the window size for the context nodes. In addition, we record the training time of each baseline for a single epoch in Table VII, where UGraphEmb is the static graph-level method, while tdGraphEmbed and GraphERT are the temporal graph-level methods. As shown in Table VII, GraphTAN is more efficient than other temporal graph-level methods in terms of both training time and computational complexity. Although UGraphEmb, as a static graph-level method, demonstrates the highest efficiency by disregarding temporal correlations, its performance on other tasks is significantly inferior to that of temporal graph-level methods. Overall, the proposed GraphTAN achieves a satisfactory balance between performance and computational complexity.

V. CONCLUSION

Learning low-dimensional graph embeddings is crucial for multiple temporal tasks, yet most existing research focuses on node-level embeddings. GraphTAN is a self-supervised, end-to-end graph-level embedding method for temporal graphs. Specifically, we propose a novel architecture based on three attention mechanisms, a practical new approach for the modeling of temporal graphs. This approach allows for the adaptive incorporation of a variety of information into the embedded learning process.

Extensive experimental results have shown that GraphTAN outperforms both static and temporal node/graph embedding methods in graph similarity and temporal anomaly detection tasks. The results of the ablation study underscore the critical role that temporal features play in learning high-quality graph embeddings. While the random walk-based methods have proven effective, they struggle with nodes with the importance that varies over time. Therefore, an end-to-end model is required to capture structural features, identify temporal patterns, and filter out temporal noise nodes. Although GraphTAN excels at modeling temporal graphs, its multiple attention modules increase training difficulty and computational complexity. Additionally, the model's generalization may be limited, requiring further adjustments when applied to different datasets and tasks. Moreover, works on training strategies for temporal graph-level embedding models remain limited. Future work could explore unsupervised approaches, such as contrastive learning and generative modeling, to train complex models more efficiently and enhance generalization.

DATA AVAILABILITY STATEMENT

The code is available at <https://github.com/Cyjcoolcool/graph-tan>.

REFERENCES

- [1] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 855–864.
- [2] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2016.
- [3] D. Wang, P. Cui, and W. Zhu, "Structural deep network embedding," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1225–1234.
- [4] K. Chakraborty, S. Bhattacharyya, and R. Bag, "A survey of sentiment analysis from social media data," *IEEE Trans. Comput. Social Syst.*, vol. 7, no. 2, pp. 450–464, Apr. 2020.
- [5] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," 2020, *arXiv:2002.07962*.
- [6] P. Goyal, S. R. Chhetri, and A. Canedo, "dyngraph2vec: Capturing network dynamics using dynamic graph representation learning," *Knowl.-Based Syst.*, vol. 187, 2020, Art. no. 104816.
- [7] U. Singer, I. Guy, and K. Radinsky, "Node embedding over temporal graphs," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4605–4612.
- [8] M. Beladev, L. Rokach, G. Katz, I. Guy, and K. Radinsky, "tdgraphemb: Temporal dynamic graph-level embedding," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 55–64.
- [9] M. Beladev, G. Katz, L. Rokach, U. Singer, and K. Radinsky, "Graphert-transformers-based temporal dynamic graph embedding," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 68–77.
- [10] Y. Bai et al., "Unsupervised inductive graph-level representation learning via graph-graph proximity," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 1988–1994.
- [11] P. Goyal, N. Kamra, X. He, and Y. Liu, "DynGEM: Deep embedding method for dynamic graphs," 2018, *arXiv:1805.11273*.
- [12] V. Miz, B. Ricaud, K. Benzi, and P. Vanderghenst, "Anomaly detection in the dynamics of web and social networks using associative memory," in *Proc. World Wide Web Conf.*, 2019, pp. 1290–1299.
- [13] D. Knoke, "Emerging trends in social network analysis of terrorism and counterterrorism," in *Emerging Trends in the Social Behavioral Sciences: An Interdisciplinary, Searchable, and Linkable Resource*, 2015, pp. 1–15.
- [14] S. Wang, J. Yang, J. Yao, Y. Bai, and W. Zhu, "An overview of advanced deep graph node clustering," *IEEE Trans. Comput. Social Syst.*, vol. 11, no. 1, pp. 1302–1314, Feb. 2024.
- [15] Y. Xu, J. Wang, M. Guang, C. Yan, and C. Jiang, "Multi-structure graph classification method with attention-based pooling," *IEEE Trans. Comput. Social Syst.*, vol. 10, no. 2, pp. 602–613, Apr. 2023.
- [16] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," 2017, *arXiv:1709.05584*.
- [17] C.-Y. Zhang, W.-P. Fang, H.-C. Cai, C. P. Chen, and Y.-N. Lin, "Sparse graph transformer with contrastive learning," *IEEE Trans. Comput. Social Syst.*, vol. 11, no. 1, pp. 892–904, Feb. 2024.
- [18] X. Luo et al., "Self-supervised graph-level representation learning with adversarial contrastive learning," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 2, pp. 1–23, 2023.
- [19] A. Narayanan, M. Chandramohan, R. Venkatesan, L. Chen, Y. Liu, and S. Jaiswal, "graph2vec: Learning distributed representations of graphs," 2017, *arXiv:1707.05005*.
- [20] D. Zhang, J. Yan, K. Polat, A. Alhudhaif, and J. Li, "Multimodal joint prediction of traffic spatial-temporal data with graph sparse attention mechanism and bidirectional temporal convolutional network," *Adv. Eng. Inform.*, vol. 62, 2024, Art. no. 102533.
- [21] A. Zhang and D. Xia, "Tensor SVD: Statistical and computational limits," *IEEE Trans. Inf. Theory*, vol. 64, no. 11, pp. 7311–7338, Nov. 2018.
- [22] F. Anowar, S. Sadaoui, and B. Selim, "Conceptual and empirical comparison of dimensionality reduction algorithms (PCA, KPCA, LDA, MDS, SVD, LLE, ISOMAP, LE, ICA, t-SNE)," *Comput. Sci. Rev.*, vol. 40, 2021, Art. no. 100378.
- [23] S. Chanpuriya, C. Musco, K. Sotiropoulos, and C. Tsourakakis, "Deep-walking backwards: from embeddings back to graphs," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2021, pp. 1473–1483.
- [24] K. W. Church, "Word2vec," *Nat. Lang. Eng.*, vol. 23, no. 1, pp. 155–162, 2017.
- [25] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [26] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [27] W. L. Hamilton, J. Leskovec, and D. Jurafsky, "Diachronic word embeddings reveal statistical laws of semantic change," 2016, *arXiv:1605.09096*.
- [28] A. Sankar, Y. Wu, L. Gou, W. Zhang, and H. Yang, "Dysat: Deep neural representation learning on dynamic graphs via self-attention networks," in *Proc. 13th Int. Conf. Web Search Data Mining*, 2020, pp. 519–527.
- [29] Z. Liu, C. Huang, Y. Yu, and J. Dong, "Motif-preserving dynamic attributed network embedding," in *Proc. Web Conf.*, 2021, pp. 1629–1638.
- [30] M. Alomrani, M. Biparva, Y. Zhang, and M. Coates, "DyG2Vec: Efficient representation learning for dynamic graphs," 2024, *arXiv:2210.16906*.
- [31] Z. Wen and Y. Fang, "Trend: Temporal event and node dynamics for graph representation learning," in *Proc. ACM Web Conf.*, 2022, pp. 1159–1169.
- [32] S. Bougleux, L. Brun, V. Carletti, P. Foggia, B. Gaüzère, and M. Vento, "Graph edit distance as a quadratic assignment problem," *Pattern Recognit. Lett.*, vol. 87, pp. 38–46, Feb. 2017.
- [33] W. Ju et al., "Unsupervised graph-level representation learning with hierarchical contrasts," *Neural Netw.*, vol. 158, pp. 359–368, Jan. 2023.
- [34] Y. Ren et al., "Hi-part: Going beyond graph pooling with hierarchical partition tree for graph-level representation learning," *ACM Trans. Knowl. Discovery Data*, vol. 18, no. 4, pp. 1–20, 2024.

- [35] H. B. Dogru, S. Tilki, A. Jamil, and A. A. Hameed, "Deep learning-based classification of news texts using doc2vec model," in *Proc. 1st Int. Conf. Artif. Intell. Data Analytics (CAIDA)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 91–96.
- [36] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [37] H.-Y. Yao, C.-Y. Zhang, Z.-L. Yao, C. P. Chen, and J. Hu, "A recurrent graph neural network for inductive representation learning on dynamic graphs," *Pattern Recognit.*, vol. 154, 2024, Art. no. 110577.
- [38] C. D. Barros, M. R. Mendonça, A. B. Vieira, and A. Ziviani, "A survey on embedding dynamic graphs," *ACM Comput. Surv. (CSUR)*, vol. 55, no. 1, pp. 1–37, 2021.
- [39] H. Gao and S. Ji, "Graph u-Nets," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2019, pp. 2083–2092.
- [40] Z. Lan, B. Hong, Y. Ma, and F. Ma, "More interpretable graph similarity computation via maximum common subgraph inference," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6588–6599, Nov. 2024.
- [41] T. N. Kipf and M. Welling, "Variational graph auto-encoders," 2016, *arXiv:1611.07308*.
- [42] L. Zhou, Y. Yang, X. Ren, F. Wu, and Y. Zhuang, "Dynamic network embedding by modeling triadic closure process," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018.
- [43] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 04, 2020, pp. 5363–5370.
- [44] B. Adhikari, Y. Zhang, N. Ramakrishnan, and B. A. Prakash, "Sub2vec: Feature learning for subgraphs," in *Proc. Adv. Knowl. Discovery Data Mining: 22nd Pacific-Asia Conf. (PAKDD)*, Melbourne, VIC, Australia: Springer, Jun. 2018, pp. 170–182.
- [45] A. Alsayat and H. El-Sayed, "Efficient genetic k-means clustering for health care knowledge discovery," in *Proc. IEEE 14th Int. Conf. Softw. Eng. Res., Manage. Appl. (SERA)*, Piscataway, NJ, USA: IEEE Press, 2016, pp. 45–52.
- [46] B. Cavallo, "Functional relations and spearman correlation between consistency indices," *J. Oper. Res. Soc.*, vol. 71, no. 2, pp. 301–311, 2020.